

07b — Integration Testing

Key Concepts

Term	Definition
Integration Test	Tests multiple components working together with real dependencies
Real Dependencies	Actual database, real API, real services (not mocks)
Test Server	A controlled server standing in for a real external API
Testing Pyramid	Strategy: many unit tests, some integration, few E2E
Regression	A previously working feature that breaks due to new changes

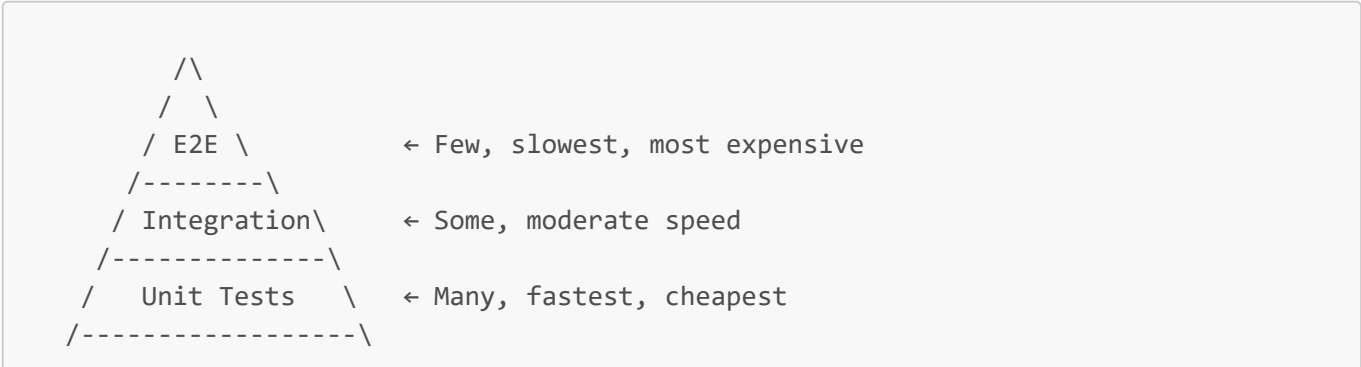
How It Works

- 1. Spin up real dependencies (test DB, test server, real services)
- 2. Seed test data if needed
- 3. Run the full flow across multiple components
- 4. Assert the combined output and side effects (DB state, notifications triggered)

Unit vs Integration — Side by Side

	Unit Test	Integration Test
Scope	One function/class	Multiple components together
Dependencies	All mocked	Real (DB, APIs, services)
Speed	Very fast (ms)	Slower (seconds)
Failure tells you	Exactly which logic broke	Which connection between components broke
Setup cost	Low	High (needs real infrastructure)
Run frequency	Every commit	Before deployment / on CI

The Testing Pyramid



- **Unit tests** — fast, cheap, pinpoint failures
 - **Integration tests** — verify components connect correctly
 - **E2E tests** — verify full user flows work end to end
-

Industry Examples

- **LinkedIn** — connection request flow tested as integration: API call → DB write → notification trigger, all together against a real test DB
 - **Amazon** — order placement integration tests verify checkout → inventory → payment → fulfilment chain before each deployment
 - **Netflix** — streaming session integration tests verify auth service → content service → billing service interact correctly
-

Interview Questions

Q1: What is integration testing? Testing that multiple components work correctly together using real dependencies — real database, real services, or test servers — rather than mocks.

Q2: How is it different from unit testing? Unit tests isolate one function with mocked dependencies. Integration tests run multiple components with real dependencies. Unit tests are faster and pinpoint exactly what broke; integration tests catch bugs at connection points between components.

Q3: Why not just write integration tests for everything? Integration tests are slow, expensive to set up, and hard to maintain. When they fail, it's harder to pinpoint which component broke. Unit tests give faster feedback and exact failure location. The Testing Pyramid guides us: many unit tests, some integration tests, few end-to-end tests.

Q4: What is the Testing Pyramid? A strategy: write many unit tests (fast, cheap), some integration tests (moderate cost), and few end-to-end tests (slow, expensive). Maximizes coverage at minimum cost.

Q5: When do integration tests typically run? Before deployment or on CI/CD pipelines — not on every commit like unit tests, because they're too slow for that frequency.